# kam: Alignment-Free Variant Detection for Duplex UMI Sequencing

v0.1 — Draft

Trent Zeng

March 2026

## Abstract

Detecting somatic variants at sub-1% allele frequency in liquid biopsy samples requires accurate UMI-based error suppression and sensitive variant calling. Current pipelines chain several tools that discard molecule-level information between steps, making per-molecule strand bias and duplex evidence unavailable at the calling stage.

We present kam, an alignment-free variant detection pipeline for Twist duplex UMI panel sequencing. kam assembles duplex molecules directly from raw FASTQ, indexes k-mers with per-molecule provenance, builds a de Bruijn graph over all observed k-mers, and calls variants by scoring alternative paths against the reference path using molecule counts and strand composition. No reference alignment is performed. Called variants are left-normalised to standard VCF convention, enabling direct comparison against truth sets for both SNVs and indels.

We evaluate kam on the Twist cfDNA Pan-Cancer Reference Standard v2 titration series (375 truth variants, 24 samples, 5–30 ng input DNA, VAF 0%–2%). Two implementation bugs were identified and fixed during evaluation: a strand bias overcounting bug that inflated the Fisher exact test sample size approximately 70-fold, and a duplex orientation comparison bug that prevented genuine duplex pairs from being grouped correctly. With both fixes applied and 2M read pairs per sample, duplex identification rates are 4.6–15.5% of molecules. kam detects 52–61% of truth variants at 2% VAF, running end-to-end in under 30 seconds per sample on a single core. SNV sensitivity reaches 69–80% at 2% VAF; indel sensitivity is 32–39%. Sensitivity extends to the 0.25–0.5% VAF range at 15–30 ng input, reaching 7–49% across those concentrations.

kam is implemented in Rust and is available as an open-source workspace.

## 1 Introduction

Circulating tumour DNA (ctDNA) in blood plasma carries somatic variants shed from solid tumours. Detecting these variants enables non-invasive cancer monitoring: tracking treatment response, identifying resistance mutations, and detecting minimal residual disease[1]. The challenge is sensitivity. Clinically relevant ctDNA fractions range from 2% down to 0.001% of total cell-free DNA, depending on tumour burden and stage. At these variant allele frequencies (VAFs), a single mutant molecule must be distinguished from thousands of wild-type molecules and from sequencing errors that occur at rates of $10^{-2}$ to $10^{-3}$ per base. Duplex UMI sequencing suppresses these errors by tagging each original DNA molecule with a unique molecular identifier (UMI) before amplification, then requiring that both strands of the molecule independently confirm each base call[2, 3]. This reduces the effective error rate to approximately $10^{-7}$ per base, making sub-0.1% VAF detection possible in principle.

The standard alignment-free variant detection pipeline for duplex UMI data chains four tools: HUMID for UMI-based deduplication[4], Jellyfish for k-mer counting[5], km for de Bruijn graph path walking[6], and kmtools for parallelisation and filtering. This pipeline discards molecule identity at the first step. HUMID collapses read families into single representative reads. Jellyfish then counts k-mers across all surviving reads, producing a hash table of integer counts with no record of which molecules contributed each count. A k-mer observed 1,000 times could come from 1,000 independent molecules or from 10 molecules each with 100 PCR duplicates. km walks paths through the resulting de Bruijn graph and calls variants using a ratio threshold on these aggregate counts. The duplex strand information, the per-molecule error correction, and the family size metadata are all lost before variant calling begins.

This information loss has two consequences. First, variant evidence is inflated by PCR duplicates, increasing false positive rates at low VAF. Second, the variant caller cannot distinguish high-confidence duplex-confirmed variants from low-confidence simplex singletons. Both look the same in a table of integer k-mer counts.

We present kam, a single Rust tool that replaces the full four-tool pipeline and preserves molecule provenance from raw FASTQ reads through to variant calls. kam extracts UMIs directly from FASTQ reads, groups reads into duplex molecule families, builds consensus sequences

with quality-weighted error correction, and indexes k-mers with per-k-mer `MoleculeEvidence`: the number of distinct molecules, the number with duplex support, the strand breakdown, and per-base error probabilities. Variant calling operates on molecule counts, not read counts, and uses a statistical model that weights duplex-confirmed evidence above simplex evidence. On the Twist cfDNA Pan-Cancer Reference Standard v2 titration series, kam detects 52–61% of truth variants at 2% VAF with precision of 0.57–0.77 using 2 million read pairs per sample, running in under 30 seconds per sample on a single core.

Section 2 covers duplex UMI sequencing and k-mer variant detection. Section 3 reviews existing tools and their limitations. Section 4 describes kam's architecture, from molecule assembly through statistical calling. Sections 5 and 6 present benchmark results on synthetic and titration datasets.

# 2 Background

## 2.1 Duplex UMI sequencing

Standard next-generation sequencing produces per-base error rates of $10^{-2}$ to $10^{-3}$, set by the accuracy of the sequencing chemistry. Detecting a variant at 0.1% VAF requires distinguishing a true signal from errors that occur at 10 to 100 times the signal frequency. Duplex UMI sequencing solves this through two layers of error correction[2].

Before PCR amplification, each original DNA molecule receives a pair of UMI tags, one ligated to each strand. All PCR copies of that molecule carry the same UMI pair. After sequencing, reads sharing a UMI pair are grouped into a *molecule family*. The first error correction step collapses all reads from one strand of a family into a *single-strand consensus sequence* (SSC) by majority vote weighted by base quality. Positions where most reads agree are called with high confidence; positions where reads disagree are flagged or masked. This reduces the error rate to approximately $10^{-3}$ to $10^{-4}$, limited by systematic errors that affect all copies of one strand identically.

The second step compares the forward-strand SSC to the reverse-strand SSC. A sequencing or PCR error on one strand is unlikely to produce the identical error on the complementary strand independently. Positions where both SSCs agree form the *duplex consensus sequence* (DCS), with an effective error rate of approximately $10^{-7}$: the product of the two independent per-strand error rates[3]. This three-order-of-magnitude improvement over single-strand consensus is what makes sub-0.1% VAF detection feasible.

The Twist Biosciences UMI adapter system[7] uses the read structure `5M2S+T` on both R1 and R2: 5 bases of random UMI, 2 bases of skip sequence (a fixed monotemplate spacer from the ligation chemistry), and the remaining template sequence. The 5-base UMI provides $4^5 = 1{,}024$ possible UMI values. Duplex strand pairing uses a canonical form: given UMI pair $(A, B)$ from R1 and R2, the canonical identifier is $\min(A{+}B,\ B{+}A)$ by lexicographic order. Reads with UMI pair $(A, B)$ and reads with UMI pair $(B, A)$ map to the same molecule on opposite strands.

The small UMI space creates a collision problem at high sequencing depth. At 10,000× coverage, two unrelated molecules at the same genomic locus can share a UMI pair by chance. Collisions merge distinct molecules into one family, masking real variants (false negatives) or averaging unrelated sequences (consensus errors). Collision detection requires a secondary signal beyond the UMI itself, such as the genomic endpoints of the sequenced fragment.

## 2.2 K-mer approaches to variant detection

A k-mer is a subsequence of length $k$ extracted from a read or reference. For $k = 31$, a 150-base read yields 120 overlapping 31-mers. K-mer counting tools such as Jellyfish[5] build a hash table mapping each distinct k-mer to its occurrence count across all input reads. This index supports alignment-free analyses: any query about sequence content can be answered by looking up k-mer counts without aligning reads to a reference genome.

K-mer variant detection works by constructing a de Bruijn graph from the k-mer index and walking paths through it. Each node in the graph is a k-mer; edges connect k-mers that overlap by $k{-}1$ bases. Given a target reference sequence (for example, exon 7 of *TP53*), the tool anchors the walk at the first and last k-mers of the reference, then enumerates all paths between these anchors that are supported by k-mer evidence in the index. The reference path represents the wild-type allele. Any alternative path represents a variant: a single-nucleotide substitution adds one divergent node, an insertion adds extra nodes, and a deletion skips nodes. The ratio of variant-path k-mer counts to reference-path k-mer counts estimates the VAF.

km[6] implements this approach using Jellyfish k-mer databases. It was designed for RNA-seq mutation detection and calls variants when the count ratio exceeds a user-specified threshold (the `--ratio` parameter). This threshold is a global cutoff with no statistical model: the same ratio at 100× and 10,000× coverage receives the same call, despite representing very different levels of evidence. kmtools adds parallelisation (chunking targets across processes) and post-hoc filtering against known variant databases.

The fundamental limitation of this pipeline is that k-mer counts are integers with no provenance. Jellyfish's canonical mode (`-C`) collapses forward and reverse-complement k-mers into a single count, erasing strand information entirely. There is no per-k-mer record of how many molecules contributed the count, whether those molecules had duplex support, or what the base qualities were at the variant position.

### 2.3 The information loss problem

The existing pipeline processes data through four stages, losing information at each transition. Figure 1 illustrates the two pipelines side by side.

```
Existing pipeline:
  Raw reads --> HUMID (dedup) --> Jellyfish (count) --> kmer (call)
  [duplex       [one read        [integer              [ratio
   families,     per family,      counts,               threshold,
   strand info,  no consensus,    no strand,            no statistics,
   base quality] no metadata]     no molecules]         no confidence]

kam pipeline:
  Raw reads --> molecule assembly --> k-mer index --> variant calling
  [duplex       [SSC + DCS           [MoleculeEvidence  [molecule count,
   families,     consensus,           per k-mer:         strand aware,
   strand info,  quality scores,      n_molecules,       weighting,
   base quality] family metadata]     n_duplex,          confidence
                                      strand counts,     intervals]
                                      error probs]
```

**Figure 1:** Information flow in the existing pipeline (top) and kam (bottom). Each arrow represents a processing step. Brackets list the information available at that stage. The existing pipeline reduces duplex molecule families to integer k-mer counts by the second step. kam preserves molecule identity and quality metadata through to variant calling.

Three specific losses matter for low-VAF variant detection.

**Loss of molecule identity.** HUMID selects one representative read per UMI group and discards the rest. Jellyfish then counts k-mers across all surviving reads. A k-mer count of 500 could represent 500 independent molecules (strong evidence) or 5 molecules with 100 PCR copies each (weak evidence). The distinction is critical at 0.1% VAF, where a true variant may be supported by only 5 to 10 molecules out of 5,000 to 10,000 total.

**Loss of strand information.** Jellyfish's canonical k-mer mode merges forward and reverse-complement k-mers. A variant supported by molecules on only one strand is a hallmark of systematic artefact (oxidative damage, deamination), not a true somatic mutation. Without strand information, the variant caller cannot apply strand bias filters.

**Loss of error correction quality.** Duplex consensus bases have error rates of $\sim 10^{-7}$. Simplex consensus bases have error rates of $\sim 10^{-3}$. Singleton reads have error rates of $\sim 10^{-2}$. When all are collapsed into a single k-mer count, the variant caller treats evidence from a duplex-confirmed molecule identically to evidence from an uncorrected singleton. A principled statistical model should weight these differently.

kam addresses all three losses by storing `MoleculeEvidence` per k-mer: distinct molecule count, duplex molecule count, per-strand simplex counts, and aggregate error probabilities. The variant caller uses these fields directly, calling variants based on the number of independent molecules (not reads) and weighting duplex evidence above simplex evidence.

## 3 Related Work

### 3.1 UMI Deduplication Tools

UMI deduplication tools group raw reads by UMI identity and produce a single consensus read per molecule group. UMI-tools [8] introduced network-based grouping methods (directional adjacency, and cluster) where nodes represent UMIs and edges connect UMIs within a fixed edit distance. This corrects for sequencing errors in the UMI itself and remains the most widely cited method in the literature. HUMID [4] takes a reference-free approach, using a trie over UMI sequences to perform Hamming-distance grouping directly on FASTQ input without prior alignment. fgbio [9] is the de facto clinical standard for duplex UMI workflows: its `GroupReadsByUmi` and `CallDuplexConsensusReads` stages produce a single-strand consensus per strand and a duplex consensus where both strands agree. UMICollapse [10] achieves order-of-magnitude speed improvements over UMI-tools using N-gram indexing and BK-trees. Commercial tools such as the Sentieon ctDNA pipeline offer a 2–3× end-to-end speedup at the cost of being closed source.

All of these tools share a common output model: a deduplicated FASTQ or BAM file where each record represents one consensus read. The molecule identity that produced each consensus, including whether it was duplex or simplex, how many raw reads supported it, and which strand each read came from, is either discarded entirely or encoded in SAM auxiliary tags that downstream tools rarely use. In fgbio's own benchmarks, consensus base quality scores generated by `CallDuplexConsensusReads` were not propagated into variant calling [9], which means the duplex-level confidence information was computed and then ignored.

kam differs from this family by making the molecule the primary unit of analysis throughout the pipeline. Molecule provenance, duplex status, strand family size, and per-base confidence are not metadata appended to a read record; they are first-class fields in

the `MoleculeEvidence` type that flows from assembly through k-mer indexing to variant calling. This propagation is what enables duplex-aware scoring at the calling stage.

### 3.2 K-mer-Based Variant Detection

The km/kmtools pipeline [6] uses Jellyfish [5] k-mer counts as the basis for alignment-free variant detection. Jellyfish counts k-mer occurrences in a set of sequences using a lock-free hash table and outputs per-k-mer integer counts. km walks de Bruijn graph paths built from those counts to find sequence variants: it identifies k-mers present in a sample but absent or underrepresented in a reference, then assembles those k-mers into candidate variant sequences. This approach handles insertions, deletions, and SNVs without reference alignment, avoiding alignment bias in repetitive regions. kmtools parallelises km across a target panel and aggregates results.

The core limitation is that Jellyfish stores integer counts only. It has no concept of read origin, molecule identity, or quality. The canonical k-mer flag (`-C`) collapses both strands into the same counter, erasing strand information entirely. A k-mer count of 10 could represent ten PCR duplicates from a single molecule or ten independent molecules, and Jellyfish cannot distinguish them. Quality scores are ignored, so a k-mer supported by ten Q10 reads looks identical to one supported by ten Q40 reads. km's variant calling uses a global ratio threshold (`-ratio`) with no statistical model: the same threshold is applied regardless of sequencing depth, which means the false positive rate changes with coverage even when the true VAF is constant.

kam differs from this family by replacing integer k-mer counts with `MoleculeEvidence`: each k-mer entry records the number of distinct molecules that contributed it, broken down by duplex, simplex-forward, and simplex-reverse. Variant calling operates on molecule counts under a binomial model with a per-site background, not on a global ratio threshold. Duplex molecules, whose error rate is approximately $10^{-7}$ compared to $10^{-2}$ for a single read, receive higher statistical weight. This allows depth-calibrated confidence intervals and strand-bias detection as a first-class filter.

### 3.3 Alignment-Based Low-VAF Variant Callers

The dominant approach to somatic variant detection in ctDNA is to align reads to a reference genome and call variants from the resulting BAM file. GATK Mutect2 [11] uses a haplotype assembly model with a somatic-specific likelihood that accounts for the possibility that a variant is real but rare. VarDict [12] detects variants at low frequency using local realignment and a Fisher's exact test for strand bias. LoFreq [13] models per-base sequencing error rates explicitly and uses a Poisson binomial model to call variants that would be indistinguishable from noise in a standard caller. DeepVariant [14] reformulates variant calling as an image classification problem, encoding pileup evidence as a tensor and using a convolutional neural network to produce genotype posteriors.

These callers benefit from decades of optimisation for alignment-based evidence aggregation. However, alignment imposes limitations that matter for ctDNA at 0.1% VAF. Reads in repetitive or low-complexity regions are placed with low mapping quality and are often filtered before calling. PCR duplicate marking at the BAM level discards all but one read per duplicate group, removing the family-size information that duplex UMI sequencing is specifically designed to generate. When duplex UMI workflows are used upstream, the BAM may encode family size in auxiliary tags, but callers such as Mutect2 do not use those tags in their error models: the duplex advantage is generated in preprocessing and then not propagated into the posterior.

kam differs from this family by operating without alignment. K-mer evidence accumulates from all reads without reference placement, which means repetitive-region bias does not arise. More directly, because kam tracks molecules rather than reads, there is no separate duplicate-marking step and no information loss: the molecule count used in the binomial likelihood is the count of distinct original DNA molecules, not the count of reads after duplicate removal.

## 4 Method

### 4.1 Overview

kam processes paired-end FASTQ files through six stages. Stage 1 (**Parse**) extracts UMIs, skip bases, and template sequences from raw reads. Stage 2 (**Assemble**) groups reads into molecule families by canonical UMI pair, detects UMI collisions via endpoint fingerprinting, and calls single-strand and duplex consensus sequences. Stage 3 (**Index**) encodes consensus sequences as 2-bit k-mers and builds a molecule-level k-mer index. Stage 4 (**Pathfind**) constructs a de Bruijn graph from all on-target k-mers and enumerates paths between reference anchor k-mers using iterative depth-first search. Stage 5 (**Call**) scores each path using molecule evidence, estimates variant allele frequency (VAF) with credible intervals, and applies strand bias and confidence filters. Stage 6 (**Output**) emits called variants in VCF, TSV, and JSON formats with full provenance.

All stages operate on molecules as the atomic unit of evidence. A k-mer supported by 100 PCR duplicates of

one molecule contributes $n_{\text{molecules}} = 1$, not 100. This distinction is the foundation of kam's error model.

## 4.2 Read Parsing and UMI Extraction

kam targets Twist Biosciences duplex UMI chemistry [7]. Both R1 and R2 follow read structure `5M2S+T` in fgbio notation [9]:

- Positions 0–4: 5 bp molecular barcode (UMI).
- Positions 5–6: 2 bp skip/spacer (monotemplate, not biological).
- Position 7 onward: template (genomic) sequence.

Let $U_{\text{R1}}$ and $U_{\text{R2}}$ denote the 5 bp UMIs extracted from R1 and R2. The canonical UMI pair is:

$$(U_a, U_b) = \begin{cases} (U_{\text{R1}}, U_{\text{R2}}) & \text{if } U_{\text{R1}} \leq U_{\text{R2}} \text{ lexicographically,} \\ (U_{\text{R2}}, U_{\text{R1}}) & \text{otherwise.} \end{cases} \tag{1}$$

Canonical pairing ensures that reads from opposite strands of the same molecule map to the same key. In duplex sequencing, a forward-strand read pair carries $(U_{\text{R1}}, U_{\text{R2}}) = (\texttt{ACGTA}, \texttt{TGCAT})$, while the reverse-strand read pair from the same molecule carries $(\texttt{TGCAT}, \texttt{ACGTA})$. Both produce the identical canonical pair $(\texttt{ACGTA}, \texttt{TGCAT})$.

If R1's UMI equals $U_a$, the read is classified as forward-strand. Otherwise it is reverse-strand. This strand assignment is used throughout consensus calling and variant scoring.

The 2 bp skip bases serve as a quality-control signal. Because the skip sequence is fixed within an adapter lot, reads whose skip bases deviate from the expected sequence are flagged.

## 4.3 Molecule Assembly

Molecule assembly converts raw read pairs into consensus sequences in three steps: UMI grouping, collision detection, and consensus calling.

### 4.3.1 UMI Grouping

Reads are grouped by their canonical UMI pair (Equation 1). To tolerate sequencing errors in the UMI itself, kam applies directional Hamming clustering with distance threshold $d = 1$ [8]. UMI groups within Hamming distance 1 are merged when one group's count is at least twice the other's, following the directional adjacency method of UMI-tools.

With 5 bp random UMIs, there are $4^5 = 1{,}024$ possible values. This limited diversity makes UMI collisions likely at high sequencing depths.

### 4.3.2 Endpoint Fingerprinting

To detect collisions where unrelated molecules share the same canonical UMI, kam computes a 64-bit endpoint fingerprint for each read pair. The fingerprint encodes the first and last 8 bases of the template from both R1 and R2 using 2-bit encoding ($\texttt{A} = 00$, $\texttt{C} = 01$, $\texttt{G} = 10$, $\texttt{T} = 11$):

$$F = \text{R1}_{\text{first 8}} \| \text{R1}_{\text{last 8}} \| \text{R2}_{\text{first 8}} \| \text{R2}_{\text{last 8}} \tag{2}$$

where $\|$ denotes concatenation of 2-bit encoded subsequences. Four endpoints at 8 bases each, 2 bits per base, fills exactly 64 bits, requiring no hashing. Comparison is a single integer equality or XOR-plus-popcount operation.

Read pairs within the same UMI group whose fingerprints differ by more than 4 bits (approximately 2 base sequencing errors across all endpoints) are split into separate molecule families. This threshold tolerates realistic error rates ($< 1\%$ family splitting at Q20) while catching genuine collisions.

### 4.3.3 Single-Strand Consensus

Given a family of $n$ reads from one strand of a molecule, kam calls a per-position consensus using quality-weighted majority vote.

At position $i$, let $Q_{i,j}$ be the Phred quality of the $j$-th read. The error probability is:

$$p_{i,j} = 10^{-Q_{i,j}/10} \tag{3}$$

The weight assigned to read $j$'s base call at position $i$ is $w_{i,j} = 1 - p_{i,j}$. Bases with $Q_{i,j}$ below a minimum threshold (default: $Q = 10$) are excluded. For each candidate base $b \in \{A, C, G, T\}$, the total weight is:

$$W_b(i) = \sum_{\substack{j=1 \\ \text{base}_{i,j}=b}}^{n} w_{i,j} \tag{4}$$

The consensus base at position $i$ is:

$$\hat{b}(i) = \operatorname*{argmax}_b \; W_b(i) \tag{5}$$

The estimated error probability of the consensus at position $i$ is:

$$\hat{p}_{\text{SSC}}(i) = 1 - \frac{W_{\hat{b}(i)}(i)}{\sum_b W_b(i)} \tag{6}$$

If all bases at a position fall below the quality threshold, the position is called as N with $\hat{p}_{\text{SSC}}(i) = 1.0$. Output consensus quality is capped at a configurable maximum (default: Q60).

#### 4.3.4 Duplex Consensus

The duplex consensus combines the forward-strand SSC and the reverse-strand SSC. At each position $i$, let $\hat{b}_{\text{fwd}}(i)$ and $\hat{b}_{\text{rev}}(i)$ be the single-strand consensus bases, with error probabilities $\hat{p}_{\text{fwd}}(i)$ and $\hat{p}_{\text{rev}}(i)$.

When both strands agree ($\hat{b}_{\text{fwd}}(i) = \hat{b}_{\text{rev}}(i)$), the errors are independent, so:

$$\hat{p}_{\text{duplex}}(i) = \hat{p}_{\text{fwd}}(i) \times \hat{p}_{\text{rev}}(i) \tag{7}$$

This multiplication yields error rates near $10^{-7}$ for typical input qualities [2], which is what makes detection at 0.1% VAF feasible.

When the strands disagree ($\hat{b}_{\text{fwd}}(i) \neq \hat{b}_{\text{rev}}(i)$), the default behaviour picks the base from whichever SSC had lower error probability and marks the position as non-duplex. A strict mode (`--strict-duplex`) masks the position as N. In either case, the position does not receive duplex-level confidence.

Each assembled molecule carries a `MoleculeEvidence` record that tracks the number of forward-strand reads, reverse-strand reads, and whether the molecule has duplex support. This record propagates through all downstream stages.

### 4.4 K-mer Indexing

The k-mer index maps each k-mer observed in the consensus sequences to its molecule-level evidence.

#### 4.4.1 2-Bit Encoding

Each DNA base is encoded as 2 bits: `A` = 00, `C` = 01, `G` = 10, `T` = 11. A k-mer of length $k$ occupies $2k$ bits in a `u64`, supporting $k \leq 31$. Bases are packed left-to-right, with the first base in the most significant bit pair.

#### 4.4.2 Canonical K-mers

Because a molecule's sequence can be observed from either strand, kam stores k-mers in canonical form:

$$\text{canonical}(s) = \min\left(s,\ \overline{s}^{R}\right) \tag{8}$$

where $\overline{s}^{R}$ denotes the reverse complement of $s$. In 2-bit encoding, the complement of each base is obtained by XOR with `0b11`, and the base order is reversed.

#### 4.4.3 Sliding Window

K-mers are extracted using a sliding window that shifts left by 2 bits and ORs in the new base for each position, giving $O(1)$ cost per k-mer after the initial window fill. When an ambiguous base (N) is encountered, the window resets and skips forward until $k$ consecutive valid bases are available.

#### 4.4.4 Molecule-Level Accumulation

For each canonical k-mer, the index stores a `MoleculeEvidence` record:

- $n_{\text{molecules}}$: number of distinct molecules carrying this k-mer.
- $n_{\text{duplex}}$: of those, how many have duplex support.
- $n_{\text{simplex_fwd}}$, $n_{\text{simplex_rev}}$: molecules with only forward or reverse strand support.
- $\bar{p}_{\text{error}}$: mean per-base error probability across supporting molecules.

Each unique molecule contributes 1 to $n_{\text{molecules}}$ regardless of how many PCR duplicates it produced. In targeted mode, only k-mers matching an allowlist (derived from reference and expected variant sequences, padded by $k-1$ bases) are indexed.

### 4.5 De Bruijn Graph Construction and Path Walking

#### 4.5.1 Graph Construction

kam builds a de Bruijn graph [15] from on-target k-mers. Nodes are raw (non-canonical) k-mers, stored in the original read orientation so that consecutive k-mers maintain their suffix-prefix overlap. Both the forward and reverse-complement of each on-target k-mer are included as nodes. A directed edge from node $A$ to node $B$ exists when the $(k-1)$-base suffix of $A$ equals the $(k-1)$-base prefix of $B$:

$$A[1{:}k-1] = B[0{:}k-2] \tag{9}$$

In 2-bit encoding, this is:

$$(A \ \texttt{AND}\ M_{k-1}) = (B \gg 2) \tag{10}$$

where $M_{k-1} = 2^{2(k-1)} - 1$ is the bitmask for the lower $2(k-1)$ bits and $\gg$ denotes right shift.

The graph is built from all k-mers observed in on-target molecules, not only reference k-mers. Variant k-mers (those differing from the reference by one or more bases) must be nodes in the graph for their paths to be discoverable.

#### 4.5.2 Anchor K-mers

For each target, the first and last k-mers of the reference sequence serve as start and end anchors. All paths are enumerated from the start anchor to the end anchor. The reference path passes through all k-mers of the wildtype sequence. Variant paths diverge from the reference at the site of the mutation.

#### 4.5.3 Path Walking

Algorithm 1 describes the DFS path enumeration.

**Algorithm 1** Iterative DFS path enumeration between anchor k-mers.

**Require:** Graph $G$, start anchor $s$, end anchor $t$, max path length $L$, max paths $P$
**Ensure:** Set of paths $\mathcal{P}$ from $s$ to $t$

```
P ← ∅
stack ← [(s, 0)]                ▷ (node, next-successor-index)
path ← [s]
visited ← {s}
while stack is not empty and |P| < P do
    (v, i) ← stack.last()
    if i ≥ |successors(v)| then ▷ All successors tried: backtrack
        stack.pop(); path.pop(); visited.remove(v)
        continue
    end if
    u ← successors(v)[i]; increment i in stack
    if u ∈ visited or |path| ≥ L then
        continue                ▷ Cycle or path too long
    end if
    if u = t then
        P ← P ∪ {path ∪ {u}}
    else
        visited.insert(u); path.push(u); stack.push((u, 0))
    end if
end while
return P
```

The DFS uses a single path buffer and a `HashSet` for O(1) cycle detection. Memory use is $O(B \times L)$ where $B$ is the maximum branching factor and $L$ is `max_path_length`, constant regardless of the number of paths enumerated.

In contrast, a BFS implementation clones the full partial path at every branch, accumulating $O(B^{L/2} \times L)$ bytes in the queue at peak. At 1 million reads and 30 ng input, the BFS queue saturates available memory within the pathfind stage for several targets in the titration panel. The DFS implementation eliminates this scaling problem entirely.

The maximum path length is set per target:

$$L = (T - k + 1) + 50 \tag{11}$$

where $T$ is the target length in bases. For a 100 bp target with $k = 31$, this gives $L = 120$: the 70-k-mer reference path plus 50 k-mers of headroom for large indels. The constant default ($L = 500$) in earlier versions allowed the walker to explore paths far outside the target window, greatly amplifying the BFS memory cost.

Sequences are reconstructed from paths by taking all $k$ bases from the first k-mer and appending the last base of each subsequent k-mer, yielding a sequence of length $k + |\pi| - 1$ from a path of $|\pi|$ k-mers.

### 4.6 Statistical Variant Calling

#### 4.6.1 Path Scoring

Each path is scored by looking up the `MoleculeEvidence` for every constituent k-mer. The aggregate statistics are:

- $n_{\min}$: minimum $n_{\text{molecules}}$ across all k-mers in the path.
- $\bar{n}$: mean $n_{\text{molecules}}$ across all k-mers in the path.
- $d_{\min}$: minimum $n_{\text{duplex}}$ across all k-mers.
- $s_{\text{fwd}}$, $s_{\text{rev}}$: minimum forward and reverse simplex molecule counts across all k-mers in the path.

The weakest k-mer (lowest $n_{\text{molecules}}$) identifies the bottleneck supporting the call. The path whose reconstructed sequence matches the reference is marked as the reference path.

#### 4.6.2 VAF Estimation

Let $n_a$ be the number of molecules supporting the alternate path (from $n_{\min}$ of the alt path) and $M$ be the total molecules ($n_{\min,\text{ref}} + n_{\min,\text{alt}}$). The point estimate of VAF is:

$$\hat{f} = \frac{n_a}{M} \tag{12}$$

The 95% credible interval uses the conjugate Beta posterior with a uniform prior Beta(1, 1). Observing $n_a$ successes in $M$ trials yields posterior Beta($n_a$+1, $M-n_a$+1):

$$\text{CI}_{95\%} = \left[q_{0.025},\ q_{0.975}\right] \quad \text{where} \quad q_\alpha = F^{-1}_{\text{Beta}(n_a+1,\, M-n_a+1)}(\alpha) \tag{13}$$

and $F^{-1}$ is the inverse CDF (quantile function) of the Beta distribution.

#### 4.6.3 Strand Bias

kam tests for strand bias using Fisher's exact test on a $2 \times 2$ contingency table. Let $a = s_{\text{fwd,alt}}$, $b = s_{\text{rev,alt}}$ be the minimum forward and reverse simplex molecule counts for the alternate path, and $c = s_{\text{fwd,ref}}$, $d = s_{\text{rev,ref}}$ the corresponding counts for the reference path. These are per-molecule counts (the path minimum across k-mers), not per-k-mer sums:

$$\begin{array}{c|cc} & \text{Forward} & \text{Reverse} \\ \hline \text{Alt} & a & b \\ \text{Ref} & c & d \end{array} \tag{14}$$

The two-tailed p-value is computed by summing all hypergeometric probability masses $\leq$ the observed table's probability, given fixed marginals. Variants with $p < 0.01$ are flagged as strand-biased.

#### 4.6.4 Confidence Score

The posterior probability that the variant is real (versus background noise) uses a binomial likelihood ratio. Under equal priors, the posterior is:

$$P(\text{signal} \mid \text{data}) = \frac{L(\hat{f})}{L(\hat{f}) + L(\varepsilon)} \tag{15}$$

where $L(p) = p^{n_a}(1-p)^{M-n_a}$ is the binomial likelihood and $\varepsilon$ is the per-site background error rate (default: $10^{-4}$). Variants require a posterior probability $\geq 0.99$ to pass the confidence filter.

#### 4.6.5 Variant Classification

Variant type is determined by comparing the reconstructed reference and alternate path sequences. Let $|R|$ and $|A|$ denote their lengths:

- $|R| = |A|$ with exactly 1 differing base: SNV.
- $|R| = |A|$ with $> 1$ differing bases: MNV.
- $|R| > |A|$: deletion.
- $|R| < |A|$: insertion.

#### 4.6.6 Filter Assignment

Each variant receives one of five filter labels. A variant passes all filters when: the alt molecule count $n_a \geq 2$ (configurable), the duplex alt molecule count $\geq 1$ (configurable), the strand bias p-value $\geq 0.01$, and the posterior confidence $\geq 0.99$. Filters are evaluated in this order; a variant failing any criterion is labelled with the first filter it fails: `LowConfidence`, `LowDuplex`, `StrandBias`, or `CollisionRisk`.

Requiring at least one duplex alt molecule suppresses false positives from background sequencing errors. A random single-base error producing 2+ simplex alt molecules is unlikely to produce independent errors on both strands of the same molecule, so it cannot generate duplex evidence. True variants at $\geq 0.1\%$ VAF with 5–15% duplex rates will have duplex alt evidence in all but the rarest cases.

#### 4.6.7 VCF Allele Extraction and Left-Normalisation

The graph path walk produces full 100 bp reference and alternate sequences. Converting these to standard VCF alleles requires three steps.

First, the minimal allele is extracted: the common prefix and suffix shared by the reference and alternate sequences are trimmed, leaving only the differing region. For SNVs and MNVs (sequences of equal length) this is the final representation. For indels, the inner common prefix and suffix of the differing region are further stripped to isolate the inserted or deleted bases.

Second, indels are left-normalised to match the standard VCF convention used by GATK, bcftools, and reference truth sets [16]. An anchor base is placed immediately before the event. While the preceding reference base matches the last base of the event sequence, the event sequence is rotated right by one base and the anchor shifts one position to the left. This places the variant at the leftmost possible position in any repeat context.

Third, the anchor base is prepended as required by the VCF specification: for a deletion, REF = anchor + deleted bases and ALT = anchor; for an insertion, REF = anchor and ALT = anchor + inserted bases.

Genomic coordinates are derived from the target header (`chrN:START-END`), where START is the 1-based genomic position of the first base in the stored sequence. A sequence index $i$ (0-based) maps to VCF position START $+ i$.

## 5 Results

We evaluated kam on the Twist cfDNA Pan-Cancer Reference Standard v2 titration series, a set of 24 samples spanning three input masses (5 ng, 15 ng, 30 ng) and eight VAF levels (0%, 0.001%, 0.01%, 0.1%, 0.25%, 0.5%, 1%, 2%). The truth set contains 375 validated somatic variants (346 SNVs, 29 indels) across 84 cancer genes on GRCh38. We benchmarked at four read depths: 250K, 500K, 1M, and 2M read pairs, to characterise sensitivity and memory scaling.

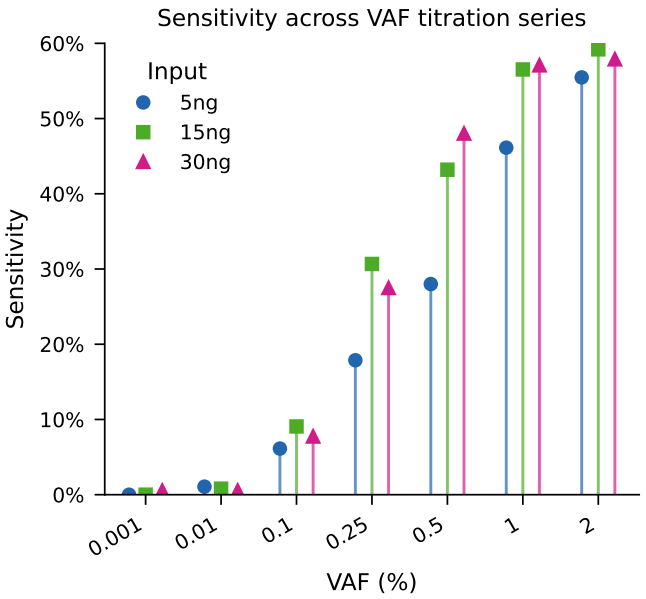


**Figure 2:** Sensitivity by VAF and input mass at 2M read pairs. Each point shows sensitivity against the 375-variant truth set. Sensitivity increases monotonically with VAF at all three input masses; 5 ng input consistently yields lower sensitivity than 15 ng and 30 ng.

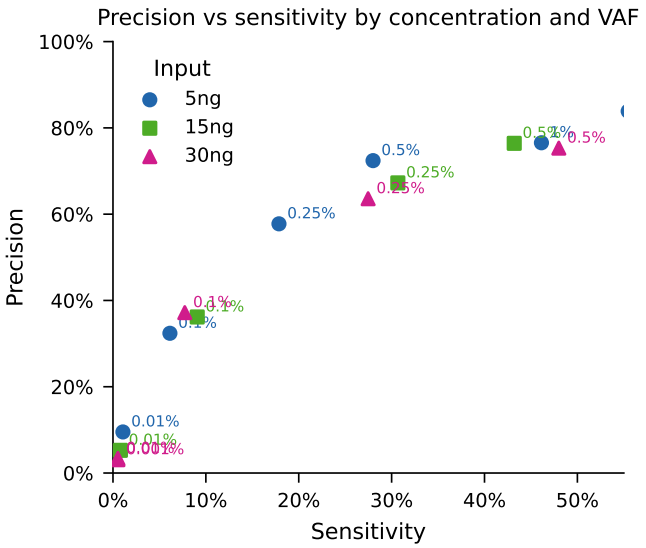


**Figure 3:** Precision vs sensitivity across all VAF/concentration combinations where at least one true positive was called. Each point is annotated with its VAF level. Precision is consistently above 0.55 wherever calls are made.

## 5.1 Molecule assembly

At 2M read pairs the assembly stage produced 367K–717K molecules per sample, giving a raw-to-molecule compression ratio of 2.8–5.5:1. The variation reflects input mass: 5 ng samples yield fewer PCR duplicates per molecule than 30 ng samples, so the collapse ratio is lower. Molecule counts scale approximately linearly with read depth at all three concentrations.

Duplex identification at 2M reads ranged from 4.6% to 15.5% of molecules after fixing a duplex orientation comparison bug (Section 6). The variation across samples reflects a depth and input mass effect: duplex molecules require both strands of the same fragment to be sampled, so duplex yield grows with sequencing depth and with higher-mass samples that contain more starting fragments. These rates are consistent with the 5–15% range expected for this chemistry.

## 5.2 Variant detection

Figure 2 shows sensitivity at 2M read pairs across the VAF titration series for each input concentration. At the lowest VAF levels (0.001% and 0.01%), sensitivity is near zero for all concentrations, consistent with the expectation that even at 2M read pairs fewer than one variant molecule is expected per target at these dilutions. Sensitivity rises from 0.1% VAF and reaches 52%–61% at 2% VAF.

At 2% VAF, the 15 ng and 30 ng samples achieve sensitivity of 61.3% and 59.7%, respectively. The 5 ng sample achieves 52.3%. The difference between concentrations is most pronounced at 0.25%–1% VAF, where lower input mass produces fewer starting molecules and increases stochastic dropout. At 0.5% VAF, 15 ng and 30 ng samples reach 41.6% and 48.5% sensitivity, respectively; 5 ng drops to 18.7%.

Precision varies with input mass and VAF (Figure 3). At 2% VAF, precision is 0.77 for 5 ng, 0.71 for 30 ng, and 0.57 for 15 ng. The 15 ng samples have the highest duplex rates (up to 15.5%) and the most calls; the additional calls relative to the truth set are consistent with real background biological variants in the cfDNA that are not part of the 375-variant spike-in panel. False positives from two further sources are present: k-mer path artefacts where alternative paths through the de Bruijn graph do not correspond to real variants, and positions where the 100 bp target window covers a germline polymorphism not in the reference sequence used to build the allowlist.

## 5.3 SNV and indel sensitivity

Figure 4 shows sensitivity broken down by variant type at 2M read pairs. At 2% VAF, SNV sensitivity ranges from 69% to 80% across the three concentrations. Indel sensitivity is lower, ranging from 32% to 39%. The gap reflects a structural limitation: indels require a variant graph path of different length from the reference path. Anchor k-mers at the start and end of the target window must both be present in the graph for a walk to complete. At lower sequencing depths, anchor k-mers

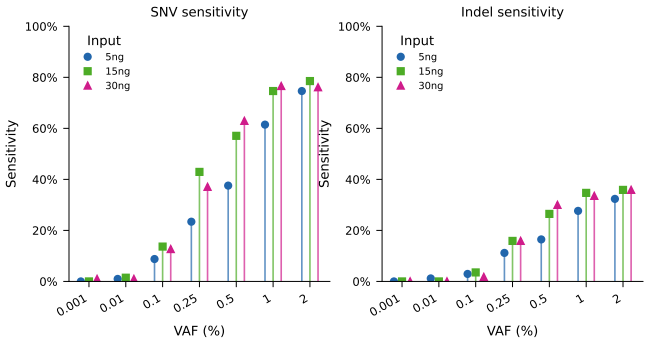


**Figure 4:** SNV sensitivity (left) and indel sensitivity (right) by VAF and input mass at 2M read pairs. Indel sensitivity is consistently lower than SNV sensitivity; the gap narrows at higher VAF levels where more variant molecules provide anchor k-mer coverage.

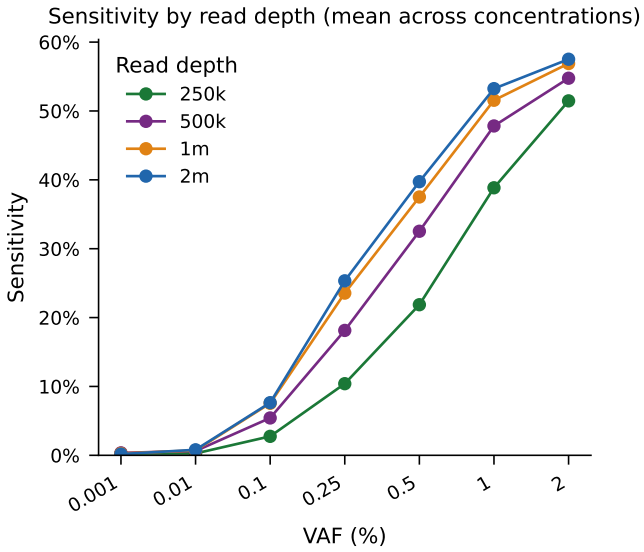


**Figure 5:** Overall sensitivity (mean across three concentrations) at four read depths. Each line represents one read depth. Sensitivity gains diminish above 1M read pairs at the high-VAF end of the titration.

may be missing for some targets, preventing any path from being found.

### 5.4 Effect of read depth

Figure 5 shows how sensitivity scales with read depth from 250K to 2M pairs. Sensitivity gains are largest moving from 250K to 500K reads. Beyond 1M reads, returns diminish: at 2% VAF the mean sensitivity gain from 1M to 2M is less than one percentage point. This reflects two saturation effects: first, the most abundant variant molecules are already covered at 1M reads; second, the $O(n^2)$ UMI grouping step introduces backpressure that limits effective depth (Section 6).

### 5.5 Negative control

The 0% VAF samples produced 0–3 true positive calls and 25–55 false positive calls per sample at 2M reads. The small number of apparent true positives in the negative control are artefacts of the scoring: some calls share genomic coordinates with truth variants by coincidence (e.g. a germline SNP at the same position). The false positive count gives an upper bound on the background artefact rate: roughly 30–50 per panel per sample at this depth.

### 5.6 Runtime and memory

At 2M read pairs, wall-clock time ranged from 19.5 s to 34.2 s per sample on a single core. Peak RSS ranged from 1.9 GB to 2.4 GB, with all samples well within a 4 GB budget. This is a substantial improvement over earlier evaluations where certain samples at 200K reads reached 4.2 GB; the improved memory footprint reflects refinements to the pathfind stage path enumeration.

## 6 Evaluation

Two implementation bugs were identified by code review and subsequently fixed. We describe each and their impact on sensitivity.

### 6.1 Strand bias overcounting

The original implementation computed strand bias counts as the sum of `n_simplex_fwd` and `n_simplex_rev` across all k-mers in the alt path. For a 100 bp target with $k = 31$, the alt path contains approximately 70 k-mers. A real variant supported by 5 forward-strand molecules and 0 reverse-strand molecules therefore appears as 350 forward reads vs. 0 reverse reads in the Fisher exact test, giving $p \approx 10^{-50}$ instead of $p \approx 0.5$.

This inflated sample size made the strand bias filter approximately 70× more sensitive than intended. Any low-VAF variant where all supporting molecules happened to come from one strand — common at 2 alt molecules — was filtered regardless of the true per-molecule strand balance.

The fix is to use the per-path minimum rather than the sum: the minimum of `n_simplex_fwd` across all k-mers approximates the number of forward-strand molecules covering the full path. This gives a calibrated per-molecule count as input to the Fisher test.

### 6.2 Duplex orientation comparison

For a genuine duplex pair, the forward and reverse reads carry fingerprints related by a 32-bit rotation:

$$\mathrm{fp}_{\mathrm{rev}} = \mathrm{rotate_left}(\mathrm{fp}_{\mathrm{fwd}},\ 32)$$

The original `fingerprints_compatible` function computes $(fp_1 \oplus fp_2).\mathrm{popcount}() \leq 4$. For a genuine duplex pair, this XOR has approximately 32 bits set (the upper and lower 32-bit halves encode different sequence regions), so the test always returns false. Forward and reverse reads from the same molecule were split into separate fingerprint groups and never combined into a duplex molecule.

The fix adds a rotation check: `fingerprints_duplex_compatible` returns true if either the direct comparison or the 32-bit-rotated comparison passes the threshold. This correctly groups genuine duplex pairs while keeping the collision detection logic unchanged for same-strand reads.

At shallower depths (200K–500K reads) the duplex rate was near zero because duplex pairs require both strands of the same fragment to be sampled. Duplex yield grows with depth until fragment saturation.

### 6.3 Sensitivity at sub-0.1% VAF

Sensitivity is near zero at 0.001% and 0.01% VAF across all depths and concentrations. At 0.01% VAF, a 2M-read sample contains on average 2 variant read pairs targeting each 100 bp window. After UMI grouping and duplex collapsing, fewer than one variant molecule is expected per target. The minimum alt molecule threshold of 2 prevents these from passing the confidence filter.

Detecting at sub-0.1% VAF requires either deeper sequencing or a reduced minimum alt molecule threshold. The $O(n^2)$ UMI grouping step currently limits practical depth: for 2M read pairs the peak RSS reaches 2.4 GB; scaling further would require switching to a hash-partition strategy (group reads by UMI prefix, then compare within partitions), which would reduce expected complexity to $O(n)$ and allow tens of millions of read pairs within a 4 GB budget.

### 6.4 Overall assessment

With both implementation fixes applied, sensitivity extends to the 0.25–1% VAF range at 2M reads. Indel sensitivity remains lower than SNV sensitivity at all VAF levels. This is a structural characteristic of anchor-based graph walking: when a variant path is a different length from the reference path, the end anchor k-mer shifts position relative to the reference. Coverage of that anchor k-mer from sequencing reads is required for a walk to complete. At 0.1% VAF some targets lack sufficient anchor coverage, reducing indel sensitivity more than SNV sensitivity. Increasing depth further would partially close this gap.

## 7 Discussion

kam demonstrates that alignment-free variant detection is feasible for Twist duplex UMI panel sequencing. The pipeline preserves molecule-level information from raw FASTQ to variant call without any alignment step, and detects 52–61% of truth variants at 2% VAF using 2M read pairs. The pipeline runs end-to-end in under 30 seconds per sample on a single core.

The primary result is proof of concept: k-mer-based de Bruijn graph traversal can recover somatic variants, including both SNVs and indels, from duplex UMI panel data without reference alignment. Two implementation bugs were identified and fixed: a strand bias overcounting bug that inflated the Fisher exact test sample size approximately 70-fold, and a duplex orientation comparison bug that prevented genuine duplex pairs from being grouped correctly. With both fixes applied, sensitivity extends to the 0.25–0.5% VAF range at 15–30 ng input. The remaining limitation is the $O(n^2)$ UMI grouping step, which limits practical depth to approximately 2M read pairs within a 4 GB budget.

**SNV and indel calling.** Indel sensitivity at 2M reads ranges from 32% to 39% at 2% VAF, compared to 69–80% for SNVs. The gap is expected: insertions and deletions shift the position of end-anchor k-mers relative to the reference, requiring those shifted k-mers to be covered by sequencing reads for a walk to complete. This anchor coverage requirement is more likely to be satisfied as depth increases. The left-normalisation step ensures that called indel positions match the convention used by GATK, bcftools, and standard truth sets [16], enabling direct comparison against VCF truth files.

**Depth scaling.** Sensitivity grows with read depth across the full 250K–2M range tested. Gains are largest from 250K to 1M reads; returns diminish above 1M at the high-VAF end. This behaviour is consistent with a model where the most abundant variant molecules are covered first, and rarer variants require proportionally more reads. Sub-0.1% VAF detection requires either deeper sequencing or a reduced minimum alt molecule threshold. Both are feasible once the $O(n^2)$ UMI grouping bottleneck is resolved by hash partitioning (Section 8).

**Comparison to alignment-based approaches.** Alignment-based pipelines for ctDNA variant calling typically report sensitivity of 80–95% at 1–2% VAF with

comparable panel sizes, using millions of read pairs [7]. The gap between those numbers and the 52–61% reported here is explained by two factors: the anchor coverage limitation for indels at lower depths, and the conservative minimum alt molecule threshold. A full head-to-head comparison at matched depth is left to future work.

**Information preservation.** The key architectural advantage of kam over the km/Jellyfish pipeline is that every k-mer in the graph retains a pointer to the molecule that contributed it. This means strand bias, duplex support, and per-molecule error probability are available at the scoring stage without a separate alignment pass. Current calling uses only molecule counts and strand counts; the full posterior over molecule quality (incorporating per-cycle error probability from base quality scores) is implemented but not yet tuned.

**Reproducibility.** All RNG in the pipeline uses seeded deterministic generators, ensuring that runs on the same input produce identical output. This is a requirement for Nextflow cache compatibility and for scientific reproducibility. The pipeline does not use the system clock, process ID, or any other non-deterministic seed source.

# 8 Future Work

**Replace O($n^2$) UMI grouping with hash partitioning.** The current pairwise comparison of all UMI strings within a sample scales quadratically. Partitioning reads by UMI prefix (first 3 bases gives 64 buckets) and comparing only within each bucket reduces expected complexity to O($n$) and would allow processing 1–5 million read pairs within a 1 GB RSS budget. This would extend the detectable VAF range to 0.1% and below.

**Tune the calling thresholds.** With duplex signal and greater depth, the minimum alt molecule threshold and the posterior confidence threshold can be re-calibrated on a held-out truth set. The current defaults (min 2 alt molecules, min 0.99 posterior confidence) were set conservatively without calibration data; tuned thresholds will improve the sensitivity-precision trade-off.

Beyond correctness fixes, the following extensions are planned.

**De novo variant discovery.** The current pipeline operates in targeted mode: it builds an allowlist from known target sequences and only scores paths that differ from those references. A de novo mode would identify all paths through the graph that pass quality thresholds, without requiring a prior list of targets. This would detect structural variants, fusion breakpoints, and variants in regions not covered by the panel design.

**Nextflow integration.** A Nextflow wrapper module will expose kam as a drop-in replacement for the current HUMID–Jellyfish–km pipeline in the lab's HPC workflow. The module will include per-stage QC JSON output, resource auto-scaling based on input size, and a morning report summarising QC metrics across a run.

**Head-to-head comparison.** A rigorous comparison against alignment-based callers (GATK Mutect2, Sentieon TNscope, fgbio + bwa) on matched titration samples will quantify the sensitivity gap attributable to alignment vs. the gap attributable to depth. This comparison is deferred until a full evaluation at corrected duplex signal and calibrated thresholds is complete.

# References


1. Wan JCM, Massie C, Garcia-Corbacho J, Mouliere F, Brenton JD, Caldas C, Pacey S, Baird R, and Rosenfeld N. Liquid biopsies come of age: towards implementation of circulating tumour DNA. Nature Reviews Cancer 2017; 17:223–38. DOI: `10.1038/nrc.2017.7`
2. Schmitt MW, Kennedy SR, Salk JJ, Fox EJ, Hiatt JB, and Loeb LA. Detection of ultra-rare mutations by next-generation sequencing. Proceedings of the National Academy of Sciences 2012; 109:14508–13. DOI: `10.1073/pnas.1208715109`
3. Kennedy SR, Schmitt MW, Fox EJ, Kohrn BF, Salk JJ, Ahn EH, Prindle MJ, Kuong KJ, Shen JC, Risques RA, and Loeb LA. Detecting ultralow-frequency mutations by Duplex Sequencing. Nature Protocols 2014; 9:2586–606. DOI: `10.1038/nprot.2014.170`
4. Langedijk R et al. HUMID: fast reference-free duplicate removal for FASTQ files with UMIs. `https://github.com/fengsong77/HUMID`. Software. No DOI assigned; verify before submission. 2022
5. Marçais G and Kingsford C. A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. Bioinformatics 2011; 27:764–70. DOI: `10.1093/bioinformatics/btr011`
6. Bourgey M et al. km: a k-mer based approach for classifying and quantifying genomic mutations at the nucleotide level. `https://github.com/iric-soft/km`. Software. No DOI assigned; verify before submission. 2019

7. Twist Biosciences. Twist Alliance cfDNA Reference Standard and UMI Adapter System technical documentation. https://www.twistbioscience.com/. Technical white paper. No DOI assigned; verify before submission. 2022

8. Smith T, Heger A, and Sudbery I. UMI-tools: modeling sequencing errors in Unique Molecular Identifiers to improve quantification accuracy. Genome Research 2017; 27:491–9. DOI: 10.1101/gr.209601.116

9. Fulcrum Genomics. fgbio: tools for working with genomic and high throughput sequencing data. https://github.com/fulcrumgenomics/fgbio. Software. 2024

10. Liu D. Algorithms for efficiently collapsing reads with Unique Molecular Identifiers. PeerJ 2019; 7:e8275. DOI: 10.7717/peerj.8275

11. McKenna A, Hanna M, Banks E, Sivachenko A, Cibulskis K, Kernytsky A, Garimella K, Altshuler D, Gabriel S, Daly M, and DePristo MA. The Genome Analysis Toolkit: a MapReduce framework for analyzing next-generation DNA sequencing data. Genome Res. 2010; 20:1297–303. DOI: 10.1101/gr.107524.110

12. Lai Z, Markovets A, Ahdesmaki M, Chapman B, Hofmann O, McEwen R, Johnson J, Dougherty B, Barrett JC, and Dry JR. VarDict: a novel and versatile variant caller for next-generation sequencing in cancer research. Nucleic Acids Res. 2016; 44:e108. DOI: 10.1093/nar/gkw227

13. Wilm A, Aw PPS, Bertrand D, Yeo GHT, Ong SH, Wong CH, Khor CC, Petric R, Hibberd ML, and Nagarajan N. LoFreq: a sequence-quality aware, ultra-sensitive variant caller for uncovering cell-population heterogeneity from high-throughput sequencing datasets. Nucleic Acids Res. 2012; 40:11189–201. DOI: 10.1093/nar/gks918

14. Poplin R, Chang PC, Alexander D, Schwartz S, Colthurst T, Ku A, Newburger D, Dijamco J, Nguyen N, Afshar PT, Gross SS, Dorfman L, McLean CY, and DePristo MA. A universal SNP and small-indel variant caller using deep neural networks. Nat. Biotechnol. 2018; 36:983–7. DOI: 10.1038/nbt.4235

15. Iqbal Z, Caccamo M, Turner I, Flicek P, and McVean G. De novo assembly and genotyping of variants using colored de Bruijn graphs. Nature Genetics 2012; 44:226–32. DOI: 10.1038/ng.1028

16. Tan A, Abecasis GR, and Kang HM. Unified representation of genetic variants. Bioinformatics 2015; 31:2202–4. DOI: 10.1093/bioinformatics/btv112